Table of Contents

Table of Contents 1
Banking RAG-Based Chatbot 2
1. Project Overview 2
2. System Architecture 3
2.1 Architecture Diagram (Mermaid) 3
3. Technology Stack 3
4. Developer Onboarding 4
4.1 Prerequisites 4
4.2 Quick Start (Local Dev) 4
5. Backend Setup 4
5.1 Local (SQLite) 4
5.2 Production (MySQL profile) 5
6. Frontend Setup 5
7. Database Schema 6
7.1 Tables 6
7.2 MySQL DDL (reference) 6
8. API Documentation 7
8.1 Auth 7
8.2 Chat (RAG) 7
8.3 User Data (scoped) 8
8.4 OpenAPI (excerpt) 8
9. RAG Architecture (Detailed) 9
9.1 Document RAG 9
9.2 SQL RAG (User-Scoped Text-to-SQL) 9
9.3 RAG Flow (Mermaid Sequence) 9
9.4 Prompting Guidelines (sketch) 10
10. Security 10
10.1 JWT Flow (Mermaid Sequence) 11
11. Logging & Monitoring 11
11.1 Logback (rolling app logs) 11
11.2 Audit Log Example (JSON line) 12

11.3 Health Checks 12
12. Deployment 12
12.1 Dockerfile (Backend) 12
12.2 Dockerfile (Frontend) 13
12.3 Docker Compose (App + MySQL) 13
13. Troubleshooting 14
14. Future Enhancements 14
15. Repository Structure 14
16. CI/CD (GitHub Actions) 14
17. Testing Strategy 15
18. User Guide (Functional) 15
19. Postman Collection (outline) 16

Banking RAG-Based Chatbot

The AI-Powered Banking Support System is designed to provide a secure, grounded, and efficient AI-driven assistant for banking operations. The system combines Retrieval-Augmented Generation (RAG) with secure backend integration to deliver accurate policy responses and controlled access to live banking data.
The solution emphasizes:
· Factual accuracy through document retrieval
· Secure backend integration
· Controlled AI behavior through guardrails
· Observability and cost optimization
· Enterprise-ready architecture design

1. Project Overview

A full- stack banking assistant containing:

- **Frontend:** Angular Web App
- **Backend:** Spring Boot (Java 17)
- **Database:** SQLite (local dev) / **MySQL (recommended for production)**
- **AI:** LangChain4j + Azure OpenAI
- **RAG:** Combines Document RAG (PDF Policies) + SQL RAG (customer data)
- **Security:** JWT, data masking, SQL guardrails, audit logging
- **Observability:** Application logs + Audit logs + Health checks

Business purpose: Provide customers with instant answers about bank policies and their own accounts.

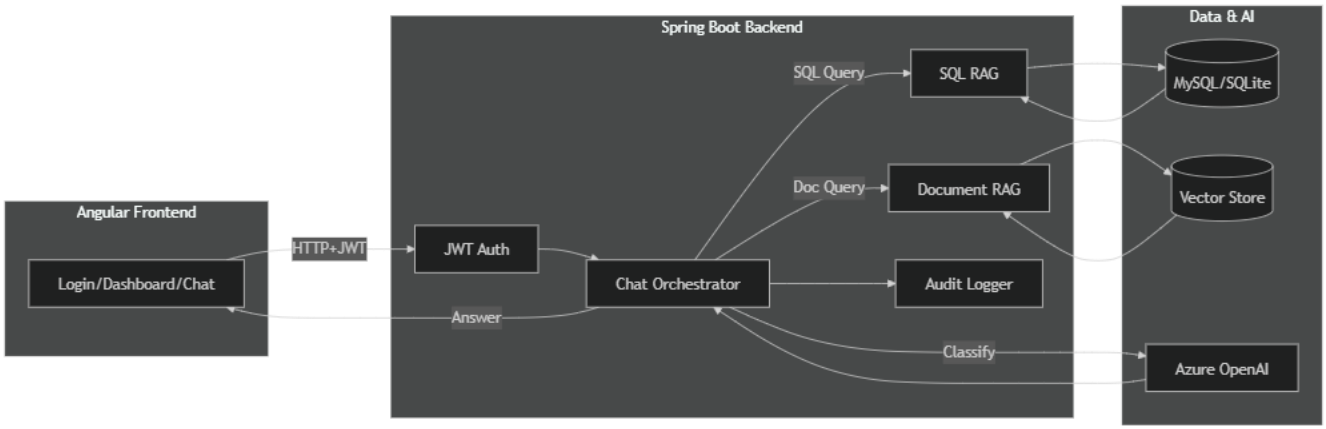
---

2. System Architecture

```
Angular (4200)
   ↓
HTTP REST (JWT)
   ↓
Spring Boot Backend (8080)
   ├── Auth Service (JWT)
   ├── Chat Service (Doc RAG + SQL RAG)
   ├── REST APIs (accounts/cards/loans)
   ↓
 MySQL / SQLite  +  Vector Store  +  Azure OpenAI

```

2.1 Architecture Diagram (Mermaid)



---

3. Technology Stack

Backend: Spring Boot, Spring Security + JWT, JPA/Hibernate, LangChain4j, Azure OpenAI SDK

Frontend: Angular 20+, TypeScript, Angular CLI

Utilities: Apache PDFBox (PDF parsing), Logback (logging)

---

4. Developer Onboarding

4.1 Prerequisites

- Java 17+, Maven 3.6+
- Node.js 18+, Angular CLI 20+
- Azure OpenAI keys and deployments

4.2 Quick Start (Local Dev)

```
# Backend
cd Backend/demo
mvn spring-boot:run
# Frontend
cd Frontend/Banking_Frontend
npm i
ng serve

```

5. Backend Setup

5.1 Local (SQLite)

Update src/main/resources/application.yml:

```
spring:
  datasource:
    url: jdbc:sqlite:bankingdb.db
    driver-class-name: org.sqlite.JDBC
  jpa:
    hibernate:
      ddl-auto: none
server:
  port: 8080
 jwt:
   secret: change_me
   expiration: 86400000   # 24h
 azure:
   openai:
     endpoint: YOUR_ENDPOINT
     key: YOUR_KEY
```

```
      deployment:
        chat: YOUR_CHAT_MODEL
        classifier: YOUR_CLASSIFIER_MODEL
  pdf:
    path: ./docs/policies.pdf
  logging:
    level:
      com.example.demo: INFO

```

5.2 Production (MySQL profile)

Create src/main/resources/application-mysql.yml:

```
spring:
  datasource:
    url:
jdbc:mysql://localhost:3306/banking?useSSL=false&allowPublicKeyRet
rieval=true&serverTimezone=UTC
    username: ${DB_USER}
    password: ${DB_PASS}
    hikari:
      maximum-pool-size: 10
  jpa:
    hibernate:
      ddl-auto: validate
    properties:
      hibernate.dialect: org.hibernate.dialect.MySQL8Dialect
jwt:
  secret: ${JWT_SECRET}
  expiration: 86400000

```

Run with profile:

```
mvn spring-boot:run -Dspring-boot.run.profiles=mysql

```

6. Frontend Setup

```
cd Frontend/Banking_Frontend
npm install
ng serve  # http://localhost:4200

```

Set API base URL in Angular service (if needed):

```
  private apiUrl = 'http://localhost:8080';

```

7. Database Schema

7.1 Tables

- **users**(id, username, password[BCrypt], roles)
- **accounts**(id, user_id, account_number, currency, balance, updated_at)
- **cards**(id, user_id, type, last4, credit_limit, outstanding, status, updated_at)
- **loans**(id, user_id, product_type, principal, interest_rate, outstanding, status, updated_at)

7.2 MySQL DDL (reference)

```
CREATE TABLE users (
   id BIGINT PRIMARY KEY AUTO_INCREMENT,
   username VARCHAR(100) NOT NULL UNIQUE,
   password VARCHAR(255) NOT NULL,
   roles VARCHAR(100) NOT NULL
);

CREATE TABLE accounts (
   id BIGINT PRIMARY KEY AUTO_INCREMENT,
   user_id BIGINT NOT NULL,
   account_number VARCHAR(32) NOT NULL UNIQUE,
   currency VARCHAR(8) NOT NULL,
   balance DECIMAL(18,2) NOT NULL DEFAULT 0,
   updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
   CONSTRAINT fk_accounts_user FOREIGN KEY (user_id) REFERENCES users(id)
);

CREATE TABLE cards (
   id BIGINT PRIMARY KEY AUTO_INCREMENT,
   user_id BIGINT NOT NULL,
   type ENUM('CREDIT','DEBIT') NOT NULL,
   last4 CHAR(4) NOT NULL,
   credit_limit DECIMAL(18,2),
   outstanding DECIMAL(18,2),
   status VARCHAR(32),
   updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
   CONSTRAINT fk_cards_user FOREIGN KEY (user_id) REFERENCES users(id)
);

CREATE TABLE loans (
   id BIGINT PRIMARY KEY AUTO_INCREMENT,
   user_id BIGINT NOT NULL,
```

```
    product_type VARCHAR(64) NOT NULL,
    principal DECIMAL(18,2) NOT NULL,
    interest_rate DECIMAL(5,2) NOT NULL,
    outstanding DECIMAL(18,2) NOT NULL,
    status VARCHAR(32),
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    CONSTRAINT fk_loans_user FOREIGN KEY (user_id) REFERENCES users(id)
);
```

```
1
```

Note (SQLite): ensure column order matches entity fields when using Hibernate with SQLite.

---

8. API Documentation

8.1 Auth

Register

```
1  POST /auth/register
2  Content-Type: application/json
3  {
4    "username": "test",
5    "password": "12345"
6  }
7
```

Login

```
1  POST /auth/login
2
```

Response → { token, username, userId }

8.2 Chat (RAG)

```
1  POST /chat/ask
2  Authorization: Bearer <token>
3  Content-Type: application/json
4  {
5    "message": "What is my balance?"
6  }
7
```

8.3 User Data (scoped)

```
GET /api/me/balance
GET /api/me/cards
GET /api/me/loans
Authorization: Bearer <token>

```

8.4 OpenAPI (excerpt)

Create src/main/resources/openapi.yaml:

```
openapi: 3.0.3
info: { title: Banking Chatbot API, version: '1.0.0' }
servers: [{ url: http://localhost:8080 }]
paths:
  /auth/login:
    post:
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/Creds' }
      responses: { '200': { description: OK } }
  /chat/ask:
    post:
      security: [{ bearerAuth: [] }]
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/ChatReq' }
      responses: { '200': { description: OK } }
components:
  securitySchemes:
    bearerAuth: { type: http, scheme: bearer, bearerFormat: JWT }
  schemas:
    Creds:
      type: object
      properties: { username: {type: string}, password: {type:
string} }
      required: [username, password]
    ChatReq:
      type: object
      properties: { message: {type: string} }
      required: [message]

```

9. RAG Architecture (Detailed)

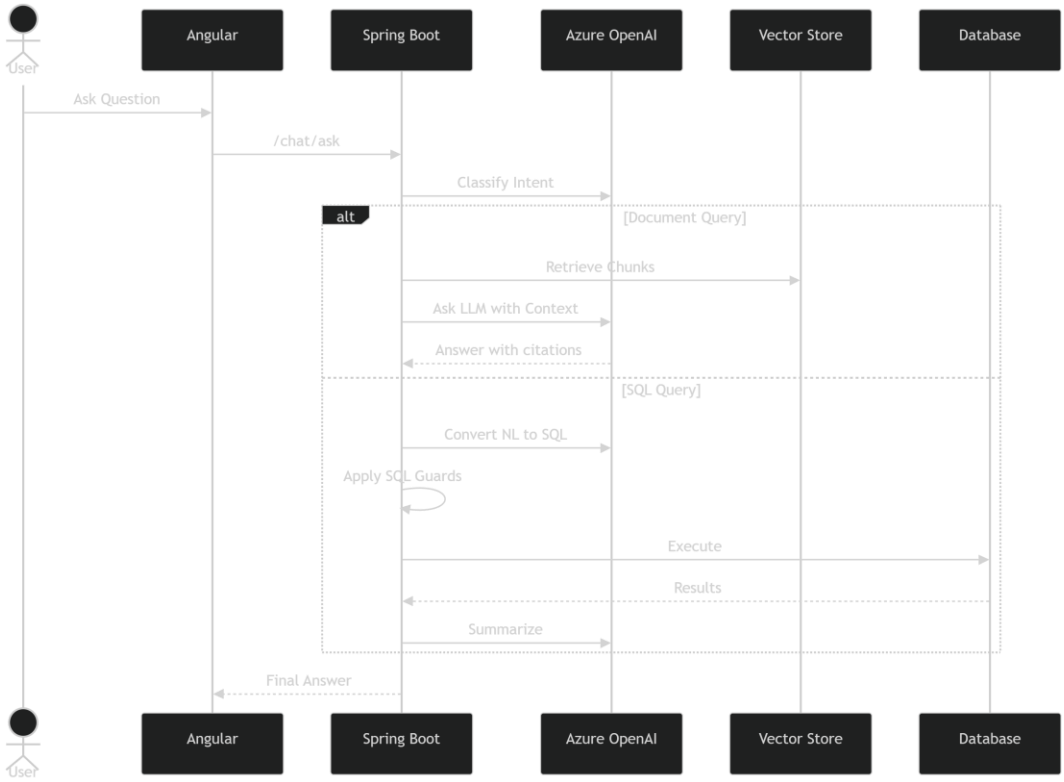
9.1 Document RAG

- **Ingestion:** PDF → text (PDFBox) → clean headers/footers → **chunk** (≈800 tokens, overlap ≈120) → embeddings → vector store.
- **Embeddings:** Start with local (e.g., AllMiniLmL6V2) or Azure embeddings for production consistency.
- **Retrieval:** Top- k (e.g., 5) with semantic similarity; consider hybrid search if store supports it.
- **Answering:** Grounded responses with citations (document titles/sections).

9.2 SQL RAG (User- Scoped Text- to- SQL)

- **Allow- list:** tables & columns; **SELECT- only**.
- **Guards:** strip comments, enforce LIMIT 100, auto- append user_id = :me.
- **Execution:** parameterized queries via JdbcTemplate/JPA.
- **Response:** mask account numbers (last4 only), normalize currency/amounts.

9.3 RAG Flow (Mermaid Sequence)



9.4 Prompting Guidelines (sketch)

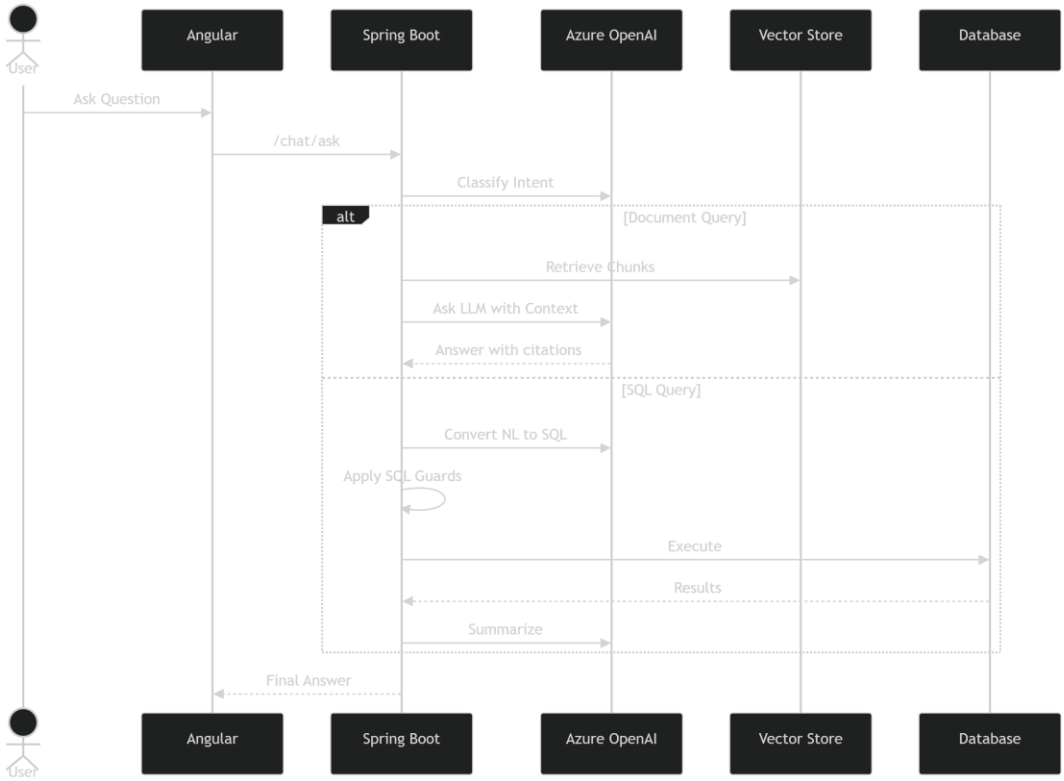
- **System:** You are a banking assistant. Respect user scope. Mask sensitive data (show last4). Refuse unrelated or operational requests. Cite sources for policy answers.
- **Classifier:** Classify as POLICY_DOC, USER_SQL, or REFUSE with brief rationale.

---

10. Security

- **AuthN/Z:** JWT (HS256), 24h expiry; BCrypt password storage; stateless sessions.
- **User Scoping:** enforce userId on all /api/me/** and SQL RAG queries.
- **SQL Safety:** block DML/DDL; strip comments; enforce LIMIT; allow- list tables/columns.
- **Data Masking:** show only last4 for account/card numbers; no PAN/secret leakage.
- **Prompt- Injection Defense:** detect out- of- scope, secrets exfiltration attempts; instruct LLM to refuse.
- **Audit Logging:** record auth attempts, chat intents, SQL statements (sanitized), response times, and trace IDs.

10.1 JWT Flow (Mermaid Sequence)



---

11. Logging & Monitoring

11.1 Logback (rolling app logs)

```
1   <configuration>
2     <appender name="FILE"
class="ch.qos.logback.core.rolling.RollingFileAppender">
3       <file>logs/application.log</file>
4       <encoder>
5         <pattern>%d %-5level [%X{traceId}] %logger
- %msg%n</pattern>
```

```
      </encoder>
      <rollingPolicy
class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
        <fileNamePattern>logs/application-%d{yyyy-MM-
dd}.log</fileNamePattern>
      </rollingPolicy>
    </appender>
    <root level="INFO"><appender-ref ref="FILE"/></root>
  </configuration>

```

11.2 Audit Log Example (JSON line)

```
{
  "type": "CHAT",
  "traceId": "c6e9...",
  "userId": 1,
  "intent": "USER_SQL",
  "promptTokens": 412,
  "durationMs": 864,
  "success": true
}

```

11.3 Health Checks

- /actuator/health (if enabled)
- Synthetic checks: /auth/login, /chat/ask
- Metrics: p95 latency < 2s (chat), error rate < 1%

---

12. Deployment

12.1 Dockerfile (Backend)

```
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY target/demo-0.0.1-SNAPSHOT.jar app.jar
ENV JAVA_OPTS=""
```

```
ENTRYPOINT ["sh","-c","java $JAVA_OPTS -jar app.jar"]

```

12.2 Dockerfile (Frontend)

```
FROM node:18 as build
WORKDIR /app
COPY . .
RUN npm ci && npm run build -- --configuration production
FROM nginx:alpine
COPY --from=build /app/dist/ /usr/share/nginx/html

```

12.3 Docker Compose (App + MySQL)

```
services:
  db:
    image: mysql:8
    environment:
      MYSQL_DATABASE: banking
      MYSQL_USER: bankuser
      MYSQL_PASSWORD: bankpass
      MYSQL_ROOT_PASSWORD: rootpass
    ports: ["3306:3306"]
  backend:
    build: ./Backend/demo
    environment:
      DB_USER: bankuser
      DB_PASS: bankpass
      JWT_SECRET: change_me
      AZURE_OPENAI_ENDPOINT: ${AZURE_OPENAI_ENDPOINT}
      AZURE_OPENAI_KEY: ${AZURE_OPENAI_KEY}
      AZURE_OPENAI_DEPLOYMENT_CHAT:
${AZURE_OPENAI_DEPLOYMENT_CHAT}
      AZURE_OPENAI_DEPLOYMENT_CLASSIFIER:
${AZURE_OPENAI_DEPLOYMENT_CLASSIFIER}
    ports: ["8080:8080"]
    depends_on: [db]
  frontend:
    build: ./Frontend/Banking_Frontend
    ports: ["4200:80"]
    depends_on: [backend]

```

13. Troubleshooting

- **SQLite “database locked”** → close DB Browser/other clients.
- **SQLite column order mismatch** → recreate using provided schema or migrate to MySQL.
- **JWT expired** → log in again (24h TTL).
- **CORS error** → set allowed origins in Spring Security/WebMvc config to http://localhost:4200 for dev.
- **Vector store resets on restart** → move to a persistent vector DB.

---

14. Future Enhancements

- Persistent vector DB (Azure AI Search / pgvector / Pinecone)
- Redis caching (accounts/cards/loans + embeddings)
- Conversation history & follow- ups
- Streaming responses (SSE)
- Multi- language support
- RBAC (admin, support)

---

15. Repository Structure

```
Backend/demo/
  ├── src/main/java/com/example/demo/{audit,config,controller,dto,entity,repository,security,service,tools,util}
  ├── src/main/resources/{application.yml,application-mysql.yml,openapi.yaml,logback-spring.xml}
  ├── bankingdb.db
  ├── pom.xml
  ├── logs/  audit-logs/
Frontend/Banking_Frontend/
  ├── src/app/{components,guards,models,services}
.github/workflows/
  ├── ci.yml  deploy.yml (optional)

```

---

16. CI/CD (GitHub Actions)

/.github/workflows/ci.yml

```yaml
name: CI
on: [push, pull_request]
jobs:
  backend:
    runs-on: ubuntu-latest
    defaults: { run: { working-directory: Backend/demo } }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: 'temurin', java-version: '17' }
      - run: mvn -B -DskipTests=false verify
  frontend:
    runs-on: ubuntu-latest
    defaults: { run: { working-directory: Frontend/Banking_Frontend } }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '18' }
      - run: npm ci && npm run build

```

17. Testing Strategy

- **Unit tests (backend):** services (auth, RAG routing), SQL guards, masking utilities.
- **Integration tests:** repositories (JPA), security filter chain (JWT), chat controller.
- **E2E (frontend):** Playwright/Cypress—login, dashboard data, chat paths.
- **Load tests:** Artillery/Gatling on /chat/ask → p95 < 2s.
- **Security tests:** JWT tampering, SQL injection payloads, prompt- injection strings.

---

18. User Guide (Functional)

- **Login/Register** → obtain JWT → dashboard shows **accounts, cards, loans** for the user.
- **Chat Examples:**
  - “What is my credit card outstanding?” → SQL RAG (masked last4)
  - “What is the KYC checklist?” → Doc RAG with citations

- “Increase my credit limit now” → refusal (operational action, out- of- scope)

---

19. Postman Collection (outline)

```
{
  "info": { "name": "Banking Chatbot API", "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json" },
  "item": [
    { "name": "Register", "request": { "method": "POST", "url": "{{baseUrl}}/auth/register" } },
    { "name": "Login", "request": { "method": "POST", "url": "{{baseUrl}}/auth/login" } },
    { "name": "Chat Ask", "request": { "method": "POST", "url": "{{baseUrl}}/chat/ask", "header": [{ "key": "Authorization", "value": "Bearer {{token}}" }] } },
    { "name": "My Balance", "request": { "method": "GET", "url": "{{baseUrl}}/api/me/balance", "header": [{ "key": "Authorization", "value": "Bearer {{token}}" }] } }
  ]
}
```